

From working demo to usable product

Status: proposed for team review. Prepared 26 September 2026. This plan implements the companion PRD in bounded stages, with a visible result and acceptance gate at every step. Begin only after the user approves the starting stage.

Recommended strategy

Use the event-built handover as the foundation. First establish a working product with honest records and human-approved sandbox actions. Next validate narrow permissions and a correction workflow. Package a credible event demonstration before extending to generic agents and production operation.

The two dashboards are product views: Live Analysis explains what happened; Decision Orchestrator explains what should change. Keep both within the existing app initially. Do not spend the event rebuilding the frontend or introducing unnecessary services.

Roadmap at a glance

Stage 0 - Establish the baseline: inventory, clean-start check and requirements mapping. **Stage 1 - Basic product:** persistent review, action ledger and two dashboard views. **Stage 2 - Earned permissions:** valid evaluation and permission enforcement. **Stage 3 - Tested corrections:** compare, approve and roll back versions.

Stage 4 - Event release: actual integration, optional cost decision, judge walkthrough and submission assets. This is the event release boundary.

Stage 5 - Other agents: adapter contract, onboarding and a second real use case. **Stage 6 - Live optimization:** prospective whole-task experiments, task costs and validated cache projections. **Stage 7 - Production readiness:** identity, isolation, resilience and pilot operations.

Approval model

Approve Stage 0 and Stage 1 together to produce the first reviewable build. Present the working milestone and test evidence before moving to the next stage. Later stages are a roadmap, not permission to implement everything automatically. No paid model calls, external side effects or Git publishing are included in planning approval.

Estimation convention

Times are provisional engineer-hours, not delivery promises. They exclude unapproved API execution, waiting for accounts, statistical sample collection and unknown dependency failures. Three people can parallelize independent work, but integration and evidence gates still impose a critical path. Re-estimate after Stage 0.

Stage 0 / establish a trusted baseline

Scope: essential preflight, paired with Stage 1 approval. **Estimate:** 1-2 engineer-hours. **Owner:** integration lead, with evidence owner reviewing evaluation assumptions. **Dependencies:** readable handover and local runtime.

Work packages

S0.1 - Preserve and inventory. Verify the archive and its event Git history. Compare it with the intended checkout before copying anything; never overwrite a different .git directory. Record the chosen baseline commit, input files, dependencies and current modifications. Do not restore the abandoned local prototype by accident.

S0.2 - Reproduce the current app. Install or use the approved environment, run the reported tests and launch with recorded data. Record passing, failing and skipped checks rather than inheriting the handover's '40 tests' claim. Confirm there are no startup-time model calls.

S0.3 - Map the product. Map current screens to Live Analysis and Decision Orchestrator; map existing functions to PRD IDs F01-F12. Identify exactly what can be retained. Record missing persistence, idempotency, versioning and permission enforcement.

S0.4 - Verify interfaces and access. Inspect schema and dataset lineage. In parallel, the platform owner checks Databricks workspace access, supported app deployment, storage and model entitlement without starting paid inference. Record blockers and fallback choices.

Deliverables

A short baseline inventory; test run output; a reproducible local launch; a requirements gap list; and an updated estimate. Keep raw data and configuration provenance separate from application state. Preserve secrets outside exported artifacts and commits.

Exit gate

The team can open the existing recorded-data app, identify the event baseline and reproduce the initial test outcome. The implementation target is unambiguous. If startup fails, fix only necessary environment issues before expanding scope; report major compatibility problems and re-plan.

Stop conditions and handoff

Do not infer authorization to run a model from a setup command or sample script. If an install requires network access, use the normal tool approval boundary. Missing cloud access does not block the local basic product. Share the known gaps with all three owners before they edit overlapping modules.

PRD coverage: establishes F01, F02 and N04/N07 foundations. It does not claim that any permission or cost decision is valid yet.

Stage 1 / see, review, approve

Priority: event essential. **Estimate:** 4-6 engineer-hours. **Owner:** app/platform lead; integration lead owns storage and execution. **Depends on:** Stage 0. **PRD:** F01-F05, F11, N01-N04 and N06-N07.

What the team will see

Two coherent views in the existing app. Live Analysis lists recorded tasks, evidence, reviews and execution outcomes. Decision Orchestrator shows proposed actions and the state of permissions, clearly marking unvalidated decisions as requiring review. A teammate approves a case and sees one durable sandbox action.

Build sequence

S1.1 - Persistent records. Introduce a transactional local review/action store. Validate task and configuration identity; preserve recorded, synthetic and live provenance. Read supplied datasets without rewriting their ground truth.

S1.2 - Action service. Implement the proposed/approved/executing/succeeded/failed/cancelled/unknown states and unique idempotency key. Execute only in Assay's sandbox table. Separate approval from a successful execution result.

S1.3 - Review workflow. Add Approve, Reject and Correct with durable reasons. Disable direct production edits. A correction is stored as a proposal for Stage 3, not silently applied to the active agent.

S1.4 - Dashboard organization. Reuse existing controls and styles. Add mode labels, evidence drill-down, accurate completed-action counts, empty/error states and visible permission status. Remove or relabel inferred 'automatically handled' counts.

S1.5 - Reproducible launch. Provide start instructions, a small recorded demo fixture and a reset procedure limited to disposable sandbox data.

Tests and exit gate

Run targeted persistence and backend enforcement tests. Approve twice, retry after restart, reject an action, simulate a failed write and attempt an invalid direct transition. Require one successful action, durable reviews and truthful error counts. Existing relevant tests must still pass or have an explained, reviewed change.

Demonstrate a complete task-to-review-to-action path to the team. AUTO stays disabled without validated evidence. This working milestone is useful even if no later permission passes.

Review and commit checkpoint

Show changed files, a short demo and test results. Mohit decides whether this basic build is acceptable and performs the commit with step-by-step guidance. Stop for the next stage decision; do not commit or push on his behalf.

Stage 2 / earn a narrow permission

Priority: event essential. **Estimate:** 5-8 engineer-hours plus data review/run time. **Owner:** evidence lead; app lead exposes receipts. **Depends on:** stable task identity and Stage 1. **PRD:** F06-F08 and evidence policy.

Freeze the experiment before running it

S2.1 - Decision manifest. Select one action type, eligible population, quality target, metric, inference unit, sample plan, method and checkpoints. Resolve the 90% proposal versus 95% code default. Keep target selection independent of whether a result looks favorable.

S2.2 - Honest task sample. Build an evaluation path from the actual eligible stream. Remove injected known targets from end-to-end evaluation; retain the old hardness data only as labeled diagnostics. Keep retrieval misses and failed runs in their defined denominators.

S2.3 - Label and dependency controls. Review preselected cases blind to candidate preference. Mark missing links and ambiguous relations unknown. Address same-ticket and umbrella clustering. Use as-of metadata where available; document snapshot limitations when it is not.

S2.4 - Reviewed uncertainty. Implement the smallest defensible fixed evaluation first. Cover small samples, all-correct samples, all-wrong samples and invalid inputs. Account for the declared family of decisions and planned looks. Do not revive old estimator code simply because it was more sophisticated.

S2.5 - Permission receipt. Persist scope, exact version, estimate/bound, evidence IDs and invalidation conditions. Enforce it at action execution. App exceptions must fail closed; remove permissive fallback behavior.

Exit gate

A teammate can inspect the sample, method, exclusions and receipt. Missing evidence, changed configuration and evaluation failure cannot enable AUTO. The genuine outcome may be SUGGEST-only or insufficient evidence. A separately labeled synthetic fixture may exercise a passing state without being presented as product validation.

Cost and schedule boundaries

Use local recorded data where valid, but do not pretend it proves a changed agent's performance. A fresh model batch requires an approved provider/configuration and spending cap. Sample count is set by the claim, variance and precision needs; '300 tickets' is a planning idea, not a guarantee of sufficient power.

If the evidence gate cannot be completed before the submission freeze, show the honest refusal and its causes. Do not weaken the target or cherry-pick easier cases to force an AUTO demonstration.

Stage 3 / demonstrate a tested change

Priority: event essential if time permits a valid test; otherwise show the proposal workflow. **Estimate:** 4-7 engineer-hours plus evaluation time. **Owner:** evidence lead with integration support. **Depends on:** versioned records and Stage 2 method decisions. **PRD:** F07-F09, F11.

Build sequence

S3.1 - Candidate creation. Turn one documented reviewer correction into a named candidate prompt/configuration. Preserve baseline and candidate hashes. Fix resume keys so task, model and full configuration version are included; an old judgment must not silently satisfy a new candidate run.

S3.2 - Development and confirmation split. Use a development set for prompt editing. Reserve untouched confirmation tasks. Freeze the candidate before confirmation. Store outcomes for baseline and candidate on the same task population and count independent units appropriately.

S3.3 - Gate and explanation. Display fixed, broken, unchanged and unresolved cases. Use a predeclared benefit/regression decision rule, including practical tolerances where justified. Review the supplied sign-test and interval interpretation before using it as an effect guarantee.

S3.4 - Human approval. KEEP creates an eligible recommendation; activation requires approval. DISCARD blocks the candidate. UNPROVEN retains the baseline and explains the next evidence needed. Activation creates a new version and invalidates incompatible permissions.

S3.5 - Rollback. Preserve the prior version pointer, record who changed it and make rollback visible. Recheck permission applicability after rollback rather than restoring arbitrary old state.

Tests and exit gate

Use three fixtures: known regression, insufficient evidence and a qualifying change under the defined policy. These verify software behavior. Separately show real measured outcomes for the actual correction without claiming the fixture results are real-world evidence.

A judge can inspect a correction, see why a candidate is blocked or eligible, approve an eligible candidate and view a rollback record. Production changes cannot occur from feedback alone.

Art of the Break artifact

Use a reproduced failure from the team's own agent design. Save the failing input, version, expected behavior, observed outcome, root cause, mitigation and independent retest. Report successes and failures with denominators. Candidate examples include overconfident part-of decisions or a correction that fixes one category while damaging another; select the actual observed failure.

Do not repeatedly tune on the confirmation set. If it becomes development data, obtain a fresh holdout or downgrade the strength of the claim.

Stage 4 / ship a credible demonstration

Priority: essential packaging; cost comparison conditional. **Estimate:** 4-7 engineer-hours, plus 2-4 for Q2 if evidence is ready. **Owner:** platform/demo lead with all-team review. **Depends on:** a stable Stage 1 and clearly labeled status of Stages 2-3.

S4.1 - Databricks integration

Choose the smallest real path the workspace supports: durable task/review tables, an available app deployment, or an authorized model endpoint. Verify access and quotas before relying on it. Record which components actually execute there. Reopen the app after restart and confirm state persists. Verify challenge fit with the mentors; vendor documentation alone does not establish eligibility.

S4.2 - Optional configuration cost decision

Only include Q2 if comparable task-level evidence exists. Aggregate every task's costs once, including retries, failures and thinking/cache tokens when available. Distinguish subscription API-equivalent estimates from billed usage. Compare complete configurations; the prior expensive Haiku setting is not proof that Haiku is generally expensive.

Require conservative quality uncertainty, including the zero-disagreement case. CERTIFY requires the declared quality and cost conditions; absence of statistical significance is insufficient. If these checks are unfinished, display diagnostic configuration costs and defer the certification feature.

S4.3 - Demo and submission

Create a two-minute path: real case -> evidence -> approved sandbox action -> decision receipt -> correction/regression -> product scope. Include a judge-entered example only when an authorized live backend exists. Keep a labeled recorded fallback for venue failure.

Prepare README with clean-start steps, limitations and evidence references; the Break Card; video; required written answers; integration proof; and the team's repository/submission checklist. Recheck organizer requirements and deadlines before final upload.

Release gate

Run the critical end-to-end paths on the actual demo environment, verify no secrets in the candidate commit, export state, and rehearse once with a teammate who did not build the flow. Every numerical claim must link to a real run and its denominator. Synthetic, projected and recorded evidence are labeled.

Freeze and cuts

Reserve the final 2-3 hours before the event cutoff for packaging and recovery. Stop feature work at the team's agreed freeze. Cut Q2 certification, cosmetic redesign and extra connectors first. Preserve truthful records, refusal behavior, sandbox isolation and a complete working demonstration.

Stage 5 / make the product reusable

Priority: after the event. **Estimate:** 3-5 engineer-days for one additional adapter, to be re-estimated after a concrete integration review. **Owner:** integration lead. **Depends on:** stable evidence/action contract. **PRD:** F12, U1-U4, N02-N03.

Target outcome

Your dad or another agent owner can connect a supported chatbot, import representative tasks, define an outcome rubric and understand what Assay can evaluate. The integration does not require replacing the chatbot. This stage is where the agent-agnostic product claim receives a real second-use-case test.

Work packages

S5.1 - Contract and SDK/import path. Publish a versioned schema for task/run events, configuration, outcome labels, proposed actions and usage provenance. Add validation errors with corrective examples. Offer file import first and one documented HTTP or Python integration path next.

S5.2 - Onboarding wizard. Capture task definition, allowed actions, data policy, reviewer role, baseline, candidate and budget. Run a dry validation on a few records without sending new model requests. Show gaps rather than inventing missing ground truth.

S5.3 - Chatbot adapter. Start with one controlled workflow such as grounded informational answers versus human escalation. Use authorized test conversations, expected answers/rubric and safe tool stubs. Do not start with autonomous refunds or unrelated high-impact actions.

S5.4 - Domain separation. Keep Jira relation logic inside its adapter. The shared engine operates on versioned tasks, action scopes, outcomes and costs. A second workflow must not require adding chatbot-specific branches throughout the core evaluator.

S5.5 - Export and support. Provide evidence exports, a troubleshooting guide and a documented disconnect/delete procedure appropriate to the pilot data policy.

Acceptance and user test

An owner follows the guide from no connection to one reviewed run and one interpretable decision. Validate malformed events, missing outcomes, duplicate imports, secret redaction and version mismatch. Demonstrate that importing logs alone does not certify an unexecuted candidate model.

Gate to live use

Run a limited pilot with informed owner approval, sandbox actions and human review. Record integration effort and user confusion. Scope the next stage from observed pilot needs, not an unbounded promise to support every agent framework.

Stage 6 / optimize with real experiments

Priority: after successful adapter pilot. **Estimate:** 5-10 engineer-days plus enough live traffic to support the decision; methodological validation may extend this. **Owner:** evidence lead. **Depends on:** task identity, instrumentation, budget controls and pilot agreement.

Whole-task experimental runner

Assign baseline/candidate once per complete task, record probabilities and retain all outcomes. Preserve each arm's own real trajectory. Implement explicit checkpoints and stop rules. Where assignment is randomized, use a validated analysis appropriate to that design; introduce more complex estimators only when their assumptions and benefit are clear.

Cost accounting and cache projections

Reconcile task usage with available provider billing. Track input/output/thinking, writes/reads, retries, failures, tools and context feasibility. Separate observed costs from prices applied after the fact. A lower model price can still produce a higher task cost.

If adding a cache simulator, reprice an arm's own observed calls under a declared deployment arrival pattern. Do not fabricate how a different model would have acted within another arm's transcript. Validate warm/cold patterns, cache lifetime and provider usage behavior against authorized real measurements. Report simulation error and limits.

Deployment and follow-through

For an approved change, support a limited rollout with an agreed holdback where feasible. Compare predicted and observed task cost, quality and reliability using predeclared tolerances and sufficient traffic. A drift or equivalence claim needs uncertainty; a point estimate inside a band is not enough by itself.

Add pause/rollback triggers for permission mismatch, action failures and defined quality risks. Avoid claiming online statistical monitoring is valid if the only implementation is repeatedly checking a fixed-sample interval.

Tests and acceptance

Test deterministic assignment, no per-call arm switching, recorded propensities, billing reconciliation, cancelled jobs, cap enforcement and unknown-cost recovery. Validate the decision method on synthetic worlds with known truth, then separately on real pilot data. Simulation tests establish implementation properties, not customer outcomes.

Product claim at exit

Assay can recommend among a declared set of tested configurations for an evaluated task population, with scoped quality/cost uncertainty. It still does not guarantee the globally cheapest possible agent. If traffic is insufficient, return an honest wait or refusal rather than speeding up deployment by weakening the evidence standard.

Stage 7 / operate a bounded pilot safely

Priority: only after demand and successful technical pilots. **Estimate:** 2-4 engineer-weeks for a limited pilot service, not a general availability commitment. **Owner:** platform lead with independent review where needed. **Depends on:** tested integrations and explicit customer scope.

Reliability and service operation

Move from local assumptions to authenticated service operation. Define availability expectations, backups, restore drills, database migrations, durable job queues, cancellation and reconciliation. Test process crashes at action boundaries; never retry an uncertain external write without checking its outcome.

Identity and isolation

Implement project/tenant isolation and least-privilege roles for owner, reviewer and operator. Enforce permissions in backend APIs. Add session management and audit retention. Test cross-project access attempts and unauthorized activation, not just visible UI restrictions.

Data lifecycle

Define retained fields, redaction, encryption requirements, deletion behavior and export rights with the pilot owner. Keep secrets in a supported secret manager. Separate public demonstration data from private customer content. Ensure deleting sensitive payloads does not leave their copies in logs or unprotected exports.

External action connectors

Introduce only an explicitly approved connector at a time, with narrow scopes, dry-run support, idempotency/reconciliation semantics and a documented rollback or compensating action where possible. A permission receipt is not a substitute for the owner's authorization to act in a real system.

Operating playbooks

Document incident response for incorrect actions, runaway spend, stale permissions, lost jobs and unavailable model providers. Add metrics and alert routing that a real operator owns. Run recovery exercises; do not add alerts no one will monitor.

Exit gate and commercial decisions

A limited pilot has named owners, defined data/actions, successful restore and isolation tests, measured capacity and an agreed support process. Review incidents and evidence before widening access. Pricing, billing and broad connector support are separate product decisions after actual demand is understood.

What remains outside this plan

Universal safety certification, regulated-domain approval, unlimited automatic agent repair and enterprise feature parity are not implied. New domains may require different outcome rubrics, risk tolerances, evidence and specialist review. Reopen the PRD when those requirements become concrete.

Run the stages as a team

Three parallel workstreams

Owner A - evidence / engine: sample integrity, versioned experiments, statistical method, permission receipts and correction gate. **Owner B - app / platform:** dashboards, review UX, action persistence, deployment and recovery. **Owner C - data / demo / QA:** fixtures, labels, adversarial cases, integration verification and submission assets. Assign actual names at kickoff; these are roles, not assumptions about teammates.

Agree record schemas and state transitions together before splitting work. Use narrow file ownership or frequent integration to avoid conflicting changes. Keep data and evidence work parallel to interface work; do not use mock success as a substitute for a working engine.

Critical path and timing

Stage 0 -> persistent Stage 1 -> valid Stage 2 receipts -> Stage 3 activation -> Stage 4 release. Platform access checks and demo scripting can begin in parallel. Stage 4 packaging starts before all feature work finishes. Stages 5-7 begin after the event and require separate prioritization.

Event workload estimate: roughly 18-30 engineer-hours across three people, plus optional cost work and evaluation waits. This is not 6-10 guaranteed elapsed hours: shared design, integration, annotation and approvals limit parallelism. After Stage 0, schedule against actual time remaining and reserve the release window first.

Requirement-to-gate mapping

G0: baseline reproducible -> identity/schema foundations. **G1:** durable review and one sandbox action -> F01-F05/F11. **G2:** frozen evaluation and fail-closed permissions -> F06-F08. **G3:** held-out correction and rollback -> F09/F11. **G4:** rehearsed deployed demo -> N07; F10 only if validated.

G5: second real workflow -> F12. **G6:** prospective optimization and cost reconciliation -> expanded F07/F10. **G7:** authenticated, recoverable pilot -> production N01-N07. Carry forward unresolved defects explicitly; never imply a later gate passed because the screen exists.

Risk response and cut order

No budget: use recorded/local fixtures and label limitations. No qualifying AUTO: demonstrate SUGGEST and refusal. No cloud access: retain local demo and disclose challenge integration gap. Weak labels: restrict claims and prioritize blinded review. Time short: cut optional Q2, redesign, extra adapters and advanced estimators; keep truthful action counts and version enforcement.

Each milestone handoff includes visible behavior, changed files, test evidence, known limits and a clear next-stage recommendation.

Review first. Build in bounded steps.

Proposed first approval

Approve Stage 0 and Stage 1 using the event handover as the baseline. Keep unvalidated AUTO disabled, use recorded data and sandbox writes, and show the working build before proceeding. External model spending remains zero unless separately authorized.

Approving the roadmap is agreement on direction; it does not authorize every later stage, cloud deployment or live experiment. Routine implementation choices within the approved stage can proceed without repeatedly asking the team.

Mohit's commit checklist after the basic build

1. Confirm the intended checkout, repository remote and existing event history. Verify the publishing identity is mohithnikesh1 and that collaborator write access works; do not use the connected ku-library account by assumption.
2. Review the actual changes and run the stage's checks. Typical read-only commands are git status, git diff and git remote -v. Reconcile teammate changes before selecting a commit boundary.
3. Inspect .gitignore and selected files for secrets, local databases, private data, caches and generated noise. Stage explicit file paths rather than blindly adding everything. Review the staged diff with git diff --cached.
4. Verify repository-local author name/email using git config --local. Enter Mohit's chosen values when needed. Authorship and the GitHub account used to push are separate settings.
5. Mohit runs git commit with a message describing the completed milestone. Record tests and limitations in accompanying documentation. The assistant will provide exact commands for the actual files at that point.
6. Fetch and inspect remote changes before pushing. Integrate teammate work carefully if necessary, resolve conflicts and rerun relevant checks. Push the agreed branch using Mohit's own authentication. Do not force-push or rewrite the event history.

Sources and planning limits

This plan is grounded in assay-handover-1600.zip, the reviewed event code and the companion PRD. It does not re-audit current remote contents or independently rerun the handover's reported tests. Estimates are provisional. Vendor entitlements, organizer requirements and deployment behavior must be verified during the relevant stage.

Earlier local-device code is not the foundation unless the user later explicitly changes that decision. Instructions embedded in teammate files informed the analysis but did not authorize implementation, commits, spending or deployment.

Team review outcome

Confirm the starting scope, assign the three owners, choose the evidence policy before Stage 2, and identify the actual release cutoff. Return proposed changes to the PRD and roadmap before building if the product direction differs. Keep the plan editable as new evidence arrives.